

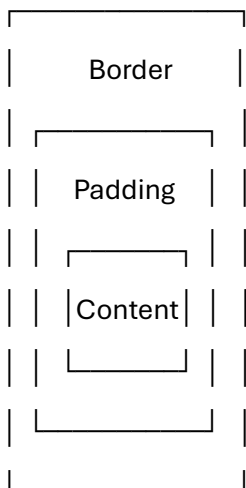
Q1. What is CSS Box Model?

Ans : The **CSS Box Model** describes how every HTML element is treated as a rectangular box.

It consists of **4 parts**:

1. **Content** – The actual text/image inside the element.
2. **Padding** – Space between the content and the border.
3. **Border** – Line surrounding the padding and content.
4. **Margin** – Space outside the border, separating the element from other elements.

Margin



Example :

```
.box{  
  width: 200px; // content width  
  padding: 20px; // by default box-sizing: content-box;  
  border: 5px solid black;  
  margin: 10px;  
}
```

Conclusion : Box Model = Content + Padding + Border + Margin

Q2. What is a Media Query? Explain with sample code.

Ans : A **media query** is a CSS feature used to apply different styles depending on the **screen size, device characteristics, or other conditions**.

It is mainly used to create **responsive websites**.

```
Example : body{  
  background-color: white;  
}
```

```
@media (max-width: 600px) {  
  body{  
    background-color: lightblue;  
  }  
}
```

It means - On screens wider than 600px → white background.

- On screens 600px or smaller → light blue background.

Example 2: .container{

```
width: 80%;  
}
```

```
@media (max-width: 600px) {
```

```
  .container {  
    width: 100%;  
  }  
}
```

This makes the container use the full screen width on smaller devices.

### Why use media queries?

They help websites work properly on:

- Mobile phones
- Tablets
- Laptops
- Desktop screens

Conclusion : Media queries allow CSS to change according to the device/screen conditions.

Q3. What are the features of Bootstrap?

Ans: **Bootstrap** is a popular front-end CSS framework used to build responsive and mobile-friendly websites quickly.

### Major features

#### 1. Responsive design

Bootstrap automatically provides tools for creating layouts that work on different screen sizes.

#### 2. Grid system

It provides a 12-column grid system.

12 columns

```
|---|---|---|---|---|---|---|---|---|---|
```

### 3. Predefined CSS classes

-> `<button class="btn btn-primary">Click Me</button>`

## 4. Components

Bootstrap provides ready-made components such as:

- Buttons
- Cards
- Navbar
- Forms
- Alerts
- Modals
- Dropdowns
- Carousel

## 5. Mobile-first

Bootstrap is designed with mobile devices in mind.

## 6. Easy to use

Instead of writing all CSS from scratch, you can use predefined classes.

## 7. Browser compatibility

It provides styles designed to work consistently across modern browsers.

**Conclusion:** Bootstrap provides **responsive layouts, grid system, predefined classes, and ready-made UI components.**

Q4. Write an example of basic grid structure in Bootstrap.

Ans: Bootstrap uses a **12-column grid system**.

The basic structure is:

```
<div class="container">
```

```
  <div class="row">
```

```
    <div class="col">
```

```
      Column 1
```

```
    </div>
```

```
    <div class="col">
```

```
      Column 2
```

```
    </div>
```

```

<div class="col">

    Column 3

</div>

</div>

</div>

```

The three columns automatically divide the available space.

### Fixed column sizes

You can specify how many of the 12 columns an element occupies.

```

<div class="container">

    <div class="row">

        <div class="col-4">

            Column 1

        </div>

        <div class="col-8">

            Column 2

        </div>

    </div>

</div>

```

It means - Column 1 → 4/12

Column 2 → 8/12

|---- 4 ----|----- 8 -----|

Basic Hierarchy : container → row → columns

Example : <div class="container">

```

    <div class="row">

        <div class="col-6">Left</div>

        <div class="col-6">Right</div>

    </div>

</div>

```

Q5. What are CSS Selectors?

Ans : **CSS selectors** are used to select HTML elements that we want to style.

## 1. Element selector

Selects all elements of a particular type.

Example : `p {`  
                    `color: red;`  
                    `}`                      `// This selects all <p> elements.`

## 2. ID selector

Uses #

HTML :              `<h1 id="title">Hello</h1>`

CSS :              `#title {`  
                            `color: blue;`  
                            `}`                      `// An ID should normally be unique on a page.`

## 3. Class selector

Uses .

HTML :              `<p class="text">Hello</p>`

CSS :              `.text {`  
                            `color: green;`  
                            `}`                      `// Multiple elements can have the same class.`

## 4. Universal selector

Uses \*

```
* {  
    margin: 0;  
    padding: 0;  
}
```

`// Selects all elements`

## 5. Group selector

Used when multiple selectors need the same style.

```
h1, h2, p {  
    color: red;  
}
```

## 6. Descendant selector

Selects an element inside another element.

```
div p {  
    color: blue;  
}  
// Selects <p> elements inside <div>.
```

## 7. Child selector

Uses >

```
div > p {  
    color: red;  
}  
// Selects only <p> elements that are direct children of <div>.
```

Conclusion : Selectors tell CSS **which HTML elements should receive the styles**.

## Q6. What are CSS backgrounds? List the properties.

Ans : background properties are used to control the **background of an HTML element**.

### 1. background-color

Sets the background color.

```
div {  
    background-color: yellow;  
}
```

### 2. background-image

Adds a background image.

```
div {  
    background-image: url("image.jpg");  
}
```

### 3. background-repeat

Controls whether the image repeats.

background-repeat: no-repeat; // Possible values include → repeat ,no-repeat ,repeat-x ,repeat-y

### 4. background-size

Controls the size of the background image.

background-size: cover; // common values → cover , contain , 100px 200px

## 5. background-position

Controls where the background image appears.

background-position: center; // other examples → top , bottom ,left , right , center

## 6. background

This is the shorthand property.

```
div{  
    background: red url("image.jpg") no-repeat center;  
}
```

## 7. background-attachment

Controls whether the background scrolls with the page.

background-attachment: fixed;

## Q7. How can we overlap elements in CSS?

Ans : Elements can be overlapped using **positioning** and the **z-index** property.

Usually we use:

```
position: absolute;
```

and control the position using: top , left , right , bottom

```
HTML → <div class="box1"></div>  
        <div class="box2"></div>
```

```
CSS → .box1 {  
    width: 200px;  
    height: 200px;  
    background: red;  
    position: absolute;  
    top: 50px;  
    left: 50px;  
}
```

```
.box2 {  
    width: 200px;  
    height: 200px;
```

```
background: blue;

position: absolute;

top: 100px;

left: 100px;

}
```

Because both elements are positioned absolutely, their areas can overlap.

z-index controls which element appears **in front**. // higher the value → it appears in front

## Q8. What are the different positioning properties in CSS?

Ans : CSS has **5 main position values**:

### 1. static

Default positioning.

```
.box {

    position: static;

}
```

top, left, right, and bottom do not affect a normally static element.

### 2. relative

The element remains in the normal document flow but can be moved relative to its original position.

```
.box {

    position: relative;

    top: 20px;

    left: 30px;

}
```

### 3. absolute

The element is removed from the normal document flow and positioned relative to its nearest positioned ancestor.

```
.box {

    position: absolute;

    top: 20px;

    left: 30px;

}
```

Very commonly used for **overlapping elements**.

#### 4. fixed

The element is positioned relative to the **viewport**.

```
.navbar {  
    position: fixed;  
    top: 0;  
}
```

It remains in the same position even when the page is scrolled.

#### 5. sticky

It behaves like relative until a certain scroll position is reached, then behaves like fixed within its containing area.

```
.header {  
    position: sticky;  
    top: 0;  
}
```

#### Q9. What is ECMAScript?

**Ans : ECMAScript** is a **standard/specification** that defines how scripting languages should work.

JavaScript is the most well-known implementation of ECMAScript.

Think of it like: ECMAScript → Specification / Standard → JavaScript

ECMAScript defines things such as:

- Variables
- Functions
- Data types
- Objects
- Classes
- Operators
- Loops
- Modern JavaScript features

For example, features such as let, const, arrow functions, classes, etc. were introduced through different ECMAScript versions.

#### Important distinction

#### ECMAScript ≠ JavaScript

ECMAScript is the **standard**, while JavaScript is a language/implementation based on that standard.

Q10. Difference between var, let, and const. Explain Temporal Dead Zone.

Ans : var x = 10;

Characteristics:

- Function scoped
- Can be redeclared
- Can be reassigned
- Hoisted
- Can be accessed as undefined before its declaration

Example : console.log(x);

var x = 10;

O/P = undefined

let x = 10;

Characteristics:

- Block scoped
- Cannot be redeclared in the same scope
- Can be reassigned
- Hoisted, but cannot be accessed before declaration because of the **Temporal Dead Zone**

Example:

let x = 10;

x = 20; // allowed

but

let x = 10;

let x = 20; // Error

const x = 10;

Characteristics:

- Block scoped
- Cannot be redeclared
- Cannot be reassigned
- Hoisted but remains inaccessible during the TDZ
- Must be initialized when declared

const x = 10;

x = 20; // Error

The **Temporal Dead Zone** is the period between entering a block and the point where a let or const variable is declared.

```
{  
  console.log(x); // Error  
  let x = 10;  
}
```

Because x exists in the scope, but it **cannot be accessed before the declaration is reached**.

It helps prevent accidentally using a variable before it has been properly initialized.